

汉奸档案 • HanJianNet 网站运维预案

基于实际探测 (IP: 47.102.207.108 , React + .NET 10 + MySQL +

Redis) 编制

编制日期：2026-09-18

一、网站现状速览

项目	当前状态	风险等级
服务器	阿里云 ECS (公网 IP 直连 , 单节点)	—
Web 服务	Nginx 反向代理 , 仅 HTTP (80端口)	● 高危
前端	React SPA (Vite 构建) , 静态文件由 Nginx 托管	● 良好
后端 API	ASP.NET Core 10 , 独立进程监听 3000 端口 (Nginx 反代)	● 关注
数据库	MySQL (推测同机部署)	● 关注
缓存	Redis (推测同机部署)	● 关注
HTTPS	未配置 , 无 SSL 证书	● 高危
域名	未绑定 (直接 IP 访问)	● 中等
安全响应头	无 (缺 CSP/HSTS/X-Frame-Options 等)	● 中等

数据量	档案条目约 899 页（数千条记录）	● 良好
用户系统	有登录/注册接口（/api/auth/login、 /api/auth/register）	● 关注
API 接口	/api/traitors、/api/traitors/{id}、/api/traitors/stats、 /api/traitors/province-stats、/api/traitors/timeline、 /api/auth/* 等	● 已识别
API 暴露	后端 3000 端口 直接暴露公网 ，可绕过 Nginx 直连	● 高危
Swagger	生产环境未开启（良好）	● 良好

二、架构总览

用户请求

|

▼

Nginx (80端口) —— 静态文件：React SPA (index.html + assets)

|

反向代理：/api → http://localhost:3000

▼

ASP.NET Core 10 (3000端口)

|

└─ MySQL（数据存储）

└─ Redis（缓存 / Session / 限流）

⚠ **重大隐患**：当前 3000 端口直接对外暴露，任何人可以跳过 Nginx 直连 .NET 后端，绕过所有 Nginx 层的安全防护（限流、WAF、HTTPS 等）。必须用防火墙封掉 3000 端口的公网访问。

三、P0 紧急事项（立刻执行，不做会出事）

1. 封掉后端端口公网访问

问题：.NET 后端 3000 端口直接暴露公网，可绕过 Nginx 所有防护。

操作：

方法一：firewalld（推荐）

```
firewall-cmd --permanent --remove-port=3000/tcp
```

firewall-cmd --permanent --remove-port=6379/tcp # Redis 也不能对外

firewall-cmd --permanent --remove-port=3306/tcp # MySQL 也不能对外

```
firewall-cmd --reload
```

方法二：阿里云安全组（最推荐，在控制台操作）

进入 ECS 安全组配置，只保留 22(SSH)、80(HTTP)、443(HTTPS) 三个端口

3306、6379、3000 全部不开放入站

验证：从外部机器 telnet 47.102.207.108 3000 应该连接失败

2. 配置 HTTPS + 域名 + ICP 备案

问题：HTTP 明文传输，登录密码可被嗅探；无域名无法正式运营；国内服务器未备案可能被封。

操作步骤：

1. **买域名：**阿里云/腾讯云，选 .com / .cn / .net，约 ¥30-100/年
2. **ICP 备案：**阿里云备案系统提交，约 7-20 天（越早越好，这是硬门槛）
3. **申请 SSL 证书：**
 - 免费：Let's Encrypt（certbot 自动续期）、阿里云免费 DV 证书
 - 付费：OV/EV 证书（适合正式运营）
4. **Nginx 配置 HTTPS（备案通过后）：**

```
server {  
  
    listen 80;  
  
    server_name yourdomain.com;  
  
    return 301 https://$server_name$request_uri; # HTTP 强制跳  
HTTPS  
}  
  
server {  
  
    listen 443 ssl http2;  
  
    server_name yourdomain.com;  
  
    ssl_certificate    /etc/nginx/ssl/fullchain.pem;
```

```
ssl_certificate_key /etc/nginx/ssl/privkey.pem;

ssl_protocols      TLSv1.2 TLSv1.3;

ssl_ciphers        HIGH:!aNULL:!MD5;

# 安全响应头

add_header X-Frame-Options "SAMEORIGIN" always;

add_header X-Content-Type-Options "nosniff" always;

add_header X-XSS-Protection "1; mode=block" always;

add_header Strict-Transport-Security "max-age=31536000;

includeSubDomains" always;

add_header Referrer-Policy "strict-origin-when-cross-origin"

always;

# 前端静态文件

location / {

    root /var/www/hanjiannet/dist;

    index index.html;

    try_files $uri $uri/ /index.html; # React SPA 路由 fallback

    # 静态资源缓存

    location ~* \.(js|css|png|jpg|jpeg|gif|ico|svg|woff2?)$ {

        expires 30d;

        add_header Cache-Control "public, immutable";

    }

}
```

```

# API 反向代理

location /api/ {

    proxy_pass      http://127.0.0.1:3000/api/;

    proxy_http_version 1.1;

    proxy_set_header Host          $host;

    proxy_set_header X-Real-IP     $remote_addr;

    proxy_set_header X-Forwarded-For

$proxy_add_x_forwarded_for;

    proxy_set_header X-Forwarded-Proto $scheme;

    # 接口超时

    proxy_connect_timeout 30s;

    proxy_read_timeout   60s;

    proxy_send_timeout    60s;

    # 限流 ( 防止 CC 攻击 )

    limit_req zone=api_limit burst=20 nodelay;

}

}

# 在 http 块中定义限流 zone ( 放在 server 外面 )

# limit_req_zone $binary_remote_addr zone=api_limit:10m

rate=10r/s;

```

3. SSH 登录加固

```

# 1. 修改 SSH 默认端口 ( 比如改成 2222 )

```

```
sed -i 's/#Port 22/Port 2222/' /etc/ssh/sshd_config
```

2. 禁止 root 密码登录（先确认密钥登录没问题再做！）

```
sed -i 's/PermitRootLogin yes/PermitRootLogin prohibit-password/'  
/etc/ssh/sshd_config
```

```
sed -i 's/#PasswordAuthentication yes/PasswordAuthentication no/'  
/etc/ssh/sshd_config
```

3. 安装 fail2ban 防暴力破解

```
yum install -y fail2ban # 或 apt install fail2ban
```

```
systemctl enable fail2ban
```

```
systemctl start fail2ban
```

4. 重启 SSHD（改端口后记得在安全组也改！）

```
systemctl restart sshd
```

⚠ 改 SSH 端口和禁密码前，**先确认密钥登录能用**，别把自己锁在外面。

四、监控告警体系

1. 分层监控清单

层级	监控项	工具/方式	告警阈值
服务器基础	CPU、内存、磁盘、带宽、存活	Netdata / 阿里云云监控	CPU > 85% 持续5分钟；磁盘 < 15%；内存 > 90%
Nginx	请求量、5xx 错误率、响应时间	GoAccess / Nginx 日志 + 脚本	5xx > 2%；请求量突增 3 倍

.NET 应用	进程存活、内存占用、GC、请求队列	dotnet-counters / 健康检查接口	进程挂了立刻报；内存 > 2GB
MySQL	连接数、慢查询、主从延迟、锁等待	mysqld_exporter / 慢查询日志	慢查询 > 500ms 且 >10条/分钟；连接数 > 80%
Redis	内存、命中率、连接数、key 数量	redis_exporter / info 命令	命中率 < 90%；内存 > 80% maxmemory
业务接口	核心接口成功率、P95 响应时间	UptimeRobot / 自建脚本	成功率 < 99%；P95 > 2s
安全	异常登录、攻击量、证书有效期	fail2ban + 邮件 / 证书到期提醒	连续失败登录 > 5 次；证书 < 30 天到期

2. 免费可用的监控方案（小型站够用）

- **UptimeRobot**（免费版）：监控网站和 API 是否存活，5 分钟一次，邮件告警
- **Netdata**：一键安装，全栈指标可视化，自带告警
- **阿里云云监控**：ECS 自带，基础监控免费

3. .NET 健康检查接口（强烈建议加）

在 Program.cs 里加：

```
builder.Services.AddHealthChecks()
```

```
.AddMySql(builder.Configuration.GetConnectionString("DefaultConnecti
on"))

.AddRedis(builder.Configuration.GetConnectionString("Redis"));

// ...

app.MapHealthChecks("/health");
```

然后在 Nginx 里允许访问 /health，监控工具轮询这个接口判断后端是否正常。

4. 日志收集与排查

日志位置	用途
/var/log/nginx/access.log	访问日志，排查攻击和流量异常
/var/log/nginx/error.log	Nginx 错误日志
/var/log/mysql/error.log或 /var/lib/mysql/*.err	MySQL 错误日志
/var/log/redis/redis.log	Redis 日志
你的 .NET 应用日志目录（如 ./logs/）	应用层错误，配合 ILogger+ Serilog/NLog

建议：.NET 用 Serilog 写文件日志，按天滚动，保留 30 天。生产环境至少 Warning 级别以上打日志，避免日志爆炸。

五、备份与灾备预案

1. 3-2-1 备份原则

- 3 份数据副本（生产 + 本地备份 + 异地备份）
- 2 种存储介质（磁盘 + 对象存储）

- 1 份异地 (阿里云 OSS / 腾讯云 COS , 与 ECS 不同地域)

2. MySQL 备份策略

每日全量备份 + 每小时 binlog 增量 :

每日凌晨 2 点全量备份 (mysqldump)

写入 /opt/backup/mysql/ 目录

```
0 2 * * * mysqldump -u root -p'你的密码' --single-transaction --  
routines --triggers \  
hanjiannet | gzip > /opt/backup/mysql/hanjiannet_$(date  
+\\%Y\\%m\\%d).sql.gz
```

保留 30 天

```
0 3 * * * find /opt/backup/mysql/ -name "*.sql.gz" -mtime +30 -  
delete
```

binlog 增量备份 (确保 MySQL 开启了 binlog)

my.cnf 中配置 :

log_bin = /var/log/mysql/mysql-bin.log

expire_logs_days = 7

更优方案 : 用 mydumper / xtrabackup 做物理备份 , 速度更快、锁更短。

3. .NET 应用与配置备份

应用发布文件 + 配置文件 + Nginx 配置

每日备份一次

```
0 3 * * * tar czf /opt/backup/app/app_$(date +%Y%m%d).tar.gz \
/opt/app/HanJianNet/ \
/etc/nginx/ \
/etc/systemd/system/hanjiannet.service \
2>/dev/null
```

4. 前端静态文件备份

前端代码有 Git 就够了，发布产物随时可以重新构建。但 index.html 和构建产物也纳入备份，省得构建环境出问题。

5. Redis 数据备份

Redis 如果只做缓存，数据丢了也没事，会自动重建。但如果存了 Session 或重要数据：

```
# Redis 开启 RDB + AOF 持久化

# redis.conf 中：

# save 900 1      # 900秒内至少1个key变化就存

# save 300 10

# save 60 10000

# appendonly yes

# appendfsync everysec
```

6. 异地备份（阿里云 OSS）

```
# 用 ossutil 把备份文件同步到 OSS
```

```
0 4 * * * ossutil cp -r /opt/backup/ oss://your-bucket/backup/${date
+\\%Y\\%m\\%d)/
```

7. 备份验证（最重要但最容易忘）

每月至少做一次恢复演练：

1. 找一台测试机（或本机的 /tmp 目录）
2. 从最新备份里恢复数据库
3. 连接 .NET 应用连测试库跑一下
4. 验证数据完整性（条数对不对、关键查询能不能出结果）

💡 没做过恢复验证的备份 = 没有备份。很多团队备份做了几年，真出事时才发现备份文件是坏的。

六、发布与回滚预案

1. 发布流程

开发完成 → Git 提交 → CI 构建 → 上传发布包 → 备份当前版本 → 停服务 → 替换文件 → 启动服务 → 验证 → 完成

建议发布时间：凌晨低峰期，避开工作日白天高峰。

2. 前端发布

本地构建

```
npm run build
```

上传到服务器（用 rsync 增量同步）

```
rsync -avz --delete dist/
```

```
user@server:/var/www/hanjiannet/dist_v20260918/
```

软链接切换（秒级切换，出问题秒回滚）

ln -sf /var/www/hanjiannet/dist_v20260918

/var/www/hanjiannet/dist

Nginx reload 一下（可选，一般不需要）

nginx -s reload

回滚：ln -sf /var/www/hanjiannet/dist_上一版本

/var/www/hanjiannet/dist，1 秒搞定。

3. .NET 后端发布

推荐用 systemd 管理服务，创建

/etc/systemd/system/hanjiannet.service：

[Unit]

Description=HanJianNet .NET API

After=network.target mysql.service redis.service

[Service]

WorkingDirectory=/opt/app/HanJianNet/current

ExecStart=/usr/bin/dotnet HanJianNet.Web.dll --

urls=http://127.0.0.1:3000

Restart=always

RestartSec=5

User=www-data

Environment=ASPNETCORE_ENVIRONMENT=Production

Environment=DOTNET_PRINT_TELEMETRY_MESSAGE=false

[Install]

WantedBy=multi-user.target

发布步骤：

1. 构建发布包（本地或 CI）

dotnet publish -c Release -o ./publish

2. 上传到服务器

rsync -avz --delete publish/

user@server:/opt/app/HanJianNet/app_v20260918/

3. 备份当前版本（如果上一版很重要）

cp -r /opt/app/HanJianNet/current /opt/app/HanJianNet/app_回

滚版本号/

4. 切换软链接 + 重启服务

ln -sf /opt/app/HanJianNet/app_v20260918

/opt/app/HanJianNet/current

systemctl restart hanjiannet

5. 验证

curl -s http://127.0.0.1:3000/health

回滚：

ln -sf /opt/app/HanJianNet/app_上一版本

/opt/app/HanJianNet/current

systemctl restart hanjiannet

4. 数据库变更（最危险）

- 所有 SQL 变更脚本**必须同时准备回滚脚本**
- 发布前在测试库先跑一遍，确认没问题
- DDL 变更（加字段、改表结构）要在业务低峰做，锁表时间长的要格外小心
- 大表加字段用 ALTER TABLE ... ALGORITHM=INPLACE, LOCK=NONE 或 pt-online-schema-change
- **发布顺序**：先改数据库（兼容老代码的变更）→ 再发布应用 → 最后做不兼容的清理

5. 发布 Checklist

- ☐ 回滚方案已就绪
- ☐ 数据库变更脚本 + 回滚脚本已准备并在测试库验证
- ☐ appsettings.json / 环境变量配置正确
- ☐ 备份已完成
- ☐ 选择低峰期发布
- ☐ 发布后验证核心接口（列表、详情、登录）
- ☐ 观察错误日志 10 分钟无异常再结束

七、故障应急处置手册

故障通用流程

发现告警 → 确认影响范围 → 快速止损（先恢复再排查）→ 排查根因 → 修复 → 复盘

场景 1：网站打不开（502 / 503）

可能原因： .NET 进程挂了 / 后端崩溃 / 数据库连不上

排查步骤：

1. 看 .NET 进程在不在

```
ps aux | grep dotnet
```

或

```
systemctl status hanjiannet
```

2. 进程挂了就重启

```
systemctl restart hanjiannet
```

3. 重启不了看应用日志

```
journalctl -u hanjiannet -n 100 --no-pager
```

或看应用自己的日志文件

```
tail -100 /opt/app/HanJianNet/current/logs/*.log
```

4. 检查数据库是否正常

```
systemctl status mysqld
```

```
mysql -u root -p -e "show processlist;"
```

5. 检查 Redis

```
redis-cli ping
```

6. 检查磁盘是否满了（最常见的坑）

```
df -h
```

常见根因：

- 磁盘满了 → 清日志、清备份

- 内存爆了被 OOM kill → dmesg | grep -i oom 查看
- 数据库连接串错了 / MySQL 挂了
- 新版本发布有 bug → 回滚到上一版本

场景 2：网站打开很慢 / 接口卡

排查步骤：

1. 看服务器资源占用

top # 看 CPU 和内存

htop # 更直观的 top

iotop # 看磁盘 IO (可能数据库在狂写)

2. 看 .NET 性能计数器 (如果装了 dotnet-counters)

dotnet-counters monitor -p \$(pgrep -f HanJianNet) \

--refresh-interval 1 \

System.Runtime[cpu-usage,working-set,gc-heap-size,threadpool-queue-length]

3. 看 MySQL 慢查询

mysqldumpslow -s t -t 10 /var/log/mysql/slow.log

或者实时看正在跑什么

mysql -u root -p -e "show full processlist;"

4. 看 Redis 情况

redis-cli info stats

redis-cli info memory

5. Nginx 看响应慢的请求

```
tail -f /var/log/nginx/access.log | awk '$NF > 1 {print}'
```

常见原因与解决：

- 某条 SQL 没走索引 → 加索引，EXPLAIN 分析
- 连接池满了 → 增大 MySQL max_connections 和 .NET 连接池

大小

- Redis 缓存击穿 → 热点数据预热，加互斥锁
- 流量突增 → 加机器 / 开 CDN / 扩 Redis 缓存

场景 3：网站被篡改 / 被挂马

应急步骤：

1. 立刻止损：

暂时关闭网站（返回维护页）

Nginx 配置改一下 root 指向维护页

或直接停 .NET 服务

```
systemctl stop hanjiannet
```

2. 修改所有密码：

- SSH 密码 / 密钥轮换
- MySQL root 密码 + 应用账号密码
- Redis 密码（如果没设的话赶紧设）
- 后台管理员密码

3. 排查入侵路径：

查最近登录记录

```
last
```

lastb # 失败的登录

查 WebShell / 可疑文件

```
find /opt/app/ -name "*.php" -o -name "*.aspx" -o -name "*.jsp"
```

.NET 站点查可疑 DLL

Nginx 日志里找异常 POST / 上传

```
grep -i "post.*upload" /var/log/nginx/access.log
```

```
grep "500\|403\|401" /var/log/nginx/access.log | tail -50
```

4. 清理 + 修复：

- 删除所有可疑文件
- 修补漏洞（上传校验、SQL 注入、权限绕过等）
- 从干净备份恢复文件和数据

5. 恢复上线 + 加固：

- 先恢复到只读状态观察
- 全面加固后再完全开放
- 接入 WAF

⚠ 如果数据库被删或被加密，**不要直接在原库上操作**，先做磁盘快照保留

现场，再用备份恢复到新库。

场景 4：CC / DDoS 攻击

判断：请求量突然暴涨，CPU/带宽打满，正常用户打不开。

应急步骤：

1. 看 Nginx 日志里 TOP IP

```
awk '{print $1}' /var/log/nginx/access.log | sort | uniq -c | sort -rn |  
head -20
```

2. 临时封 IP

```
iptables -I INPUT -s 恶意IP -j DROP
```

或用 fail2ban 自动封

3. 加大 Nginx 限流

```
# limit_req_zone $binary_remote_addr zone=api_limit:10m  
rate=5r/s;
```

4. 接入 CDN / 高防 (终极方案)

把域名切到 Cloudflare / 阿里云高防 IP

源站只允许 CDN 回源 IP 访问

长期方案：

- 接入 Cloudflare (免费版就够用) 或阿里云盾
- 源站 IP 隐藏，只允许 CDN 回源 IP 访问 80/443
- API 接口加限流 (.NET 里用 `AspNetCoreRateLimit` + Redis)
- 验证码防机器刷

场景 5：数据误删 / 误操作

应急：

1. **不要写库：**立刻停写 (只读模式或停服务)，防止覆盖数据
2. **用备份恢复：**
 - 全量备份恢复到新库
 - 用 binlog 做时间点恢复 (PITR)，恢复到删除前一秒

3. 验证数据完整性

预防：

- 数据库操作权限最小化，应用账号不给 DROP / TRUNCATE 权限
- 重要删除操作走审批
- 开启 MySQL binlog，有后悔药

场景 6：Redis 故障

Redis 挂了影响判断：

- 如果纯缓存 → 应用降级，直接查数据库（虽然慢但能用）
- 如果存 Session → 用户全部掉线，需要重新登录

排查：

redis-cli ping

systemctl status redis

tail -50 /var/log/redis/redis.log

内存爆了？

redis-cli info memory | grep used_memory_human

预防：

- .NET 里做 Redis 熔断降级，Redis 挂了自动走 DB
- 设置 maxmemory-policy allkeys-lru，内存满了自动淘汰旧 key
- 开启持久化，挂了能快速恢复数据

八、安全加固细则

1. MySQL 安全

运行安全安装脚本

mysql_secure_installation

主要做这几件事：

- 设 root 强密码

- 删除匿名用户

- 禁止 root 远程登录

- 删除 test 数据库

创建应用专用账号（别用 root 跑应用）：

CREATE USER 'hanjian_app'@'localhost' IDENTIFIED BY '强密码';

GRANT SELECT, INSERT, UPDATE, DELETE ON hanjiannet.* TO

'hanjian_app'@'localhost';

-- 不给 DROP / ALTER / TRUNCATE 权限！

FLUSH PRIVILEGES;

开启慢查询日志（my.cnf）：

slow_query_log = 1

slow_query_log_file = /var/log/mysql/slow.log

long_query_time = 0.5 # 超过 0.5 秒就算慢查询

log_queries_not_using_indexes = 1

2. Redis 安全

redis.conf 中配置：

bind 127.0.0.1 # 只允许本机访问

protected-mode yes

requirepass 你的强密码 # 设密码

rename-command FLUSHALL "" # 禁用危险命令

rename-command CONFIG ""

rename-command KEYS ""

3. .NET 应用安全

- **生产环境关闭 Swagger**：if

```
(app.Environment.IsDevelopment()) { app.UseSwagger(); }
```

- **API 限流**：用 `AspNetCoreRateLimit` 包，基于 IP + 客户端 ID 限流

- **密码哈希**：必须用 `BCrypt` 或 `Identity` 的 `PasswordHasher`，绝对不能明文或 MD5

- **JWT**：用强密钥（至少 256 位），设置合理过期时间，支持吊销
- **输入校验**：所有接口参数加 `[Required]`、正则校验，防注入
- **CORS 白名单**：别 `AllowAnyOrigin()`，只允许你的前端域名
- **CSRF 防护**：如果用 Cookie 认证，加

`ValidateAntiForgeryToken`

- **错误信息**：生产环境不返回详细异常堆栈，用 `ExceptionHandler` 统一处理

- **HTTPS 强制** : `app.UseHttpsRedirection(); app.UseHsts();`

4. Nginx 安全

- 隐藏版本号 : `server_tokens off;`
- 限制请求体大小 : `client_max_body_size 10m;`
- 禁掉不必要的 HTTP 方法 : `if ($request_method !~`

`^(GET|HEAD|POST|PUT|DELETE|OPTIONS)$ ) { return 405; }`

- 防爬虫 / 恶意 User-Agent : 可以加规则屏蔽

5. 服务器系统安全

- 定期 `yum update / apt upgrade` 打安全补丁
- 防火墙只开必要端口 (22/80/443)
- 禁止 root 远程登录
- fail2ban 防 SSH 暴力破解
- 重要操作开启审计 (`auditd`)

九、.NET 性能优化建议

根据你的技术栈，列几个容易出问题的点：

1. 数据库优化

- **EF Core 优化** :
 - 避免 N+1 查询，用 `Include / ThenInclude` 预加载
 - 分页查询必须 `Skip().Take()`，不能一次性把几千条全拉出来
 - 只读查询加 `AsNoTracking()`

- 复杂查询用 Dapper 直接写 SQL , 比 EF 快

- **索引优化** : 档案列表的筛选字段 (姓名、朝代、标签) 必须加索引

引

- **连接池** : 连接字符串里设 Max Pool Size=100 , 默认可能不够

2. Redis 缓存

- 热点数据缓存 : 档案详情、统计数据 (/stats、/province-stats、/timeline)
- 列表页按页码+筛选条件缓存
- 缓存过期时间用随机值, 防缓存雪崩
- 用 Redis 存 Session 和限流计数

3. 前端优化

- 前端已经上 CDN 的话, 静态资源走 CDN , 源站压力小很多
- React 打包用 Vite 自带的 code splitting , 首屏加载更快
- 图片压缩 + WebP 格式

十、日常运维 Checklist

每日 (5分钟)

- [] 看一眼监控大盘 : CPU、内存、磁盘、请求量
- [] 扫一眼错误日志 : Nginx error.log + .NET 错误日志
- [] 备份是否成功 (昨晚的备份文件大小是否正常)

每周 (30分钟)

- ☐ 慢查询日志分析，优化 Top 10 慢 SQL
- ☐ 检查证书和域名到期时间（提前 30 天续期）
- ☐ 看 Nginx 访问日志，有没有异常攻击
- ☐ 磁盘增长趋势评估

每月 (2小时)

- ☐ **备份恢复演练**（必做！）
- ☐ 系统安全补丁更新
- ☐ 数据库表碎片整理 / OPTIMIZE TABLE
- ☐ 账号权限审计（离职/不用的账号删掉）
- ☐ 漏洞扫描（用 OpenVAS 或阿里云安全中心）

每季度

- ☐ 全量安全审计
- ☐ 压力测试（评估当前系统能扛多少并发）
- ☐ 架构评估（是否需要升级配置 / 加机器）

十一、应急关键信息登记表

把这些信息存在一个安全的地方（密码管理器或加密文档），出问题时不用到处找：

项目	信息
服务器 IP	47.102.207.108
SSH 端口	22（建议改成别的）
SSH 密钥/密码	（此处填你的）

阿里云账号	(此处填你的)
域名	(待注册)
SSL 证书位置	/etc/nginx/ssl/
前端部署路径	/var/www/hanjiannet/dist
后端部署路径	/opt/app/HanJianNet/current
.NET 服务名	hanjiannet
MySQL root 密码	(此处填你的)
MySQL 应用账号	hanjian_app / (密码)
Redis 密码	(此处填你的)
备份目录	/opt/backup/
异地备份 OSS	bucket 名 + 密钥
日志位置	Nginx: /var/log/nginx/ ; MySQL: /var/log/mysql/ ; .NET: 应用目录/logs/
紧急联系人	(你的手机号 + 备用联系人)
云服务商工单/客服电话	阿里云 : 95187

十二、按优先级排序的行动计划

优先级	事项	预计耗时
● P0	封掉 3000、3306、6379 的公网访问 (安全组/防火墙)	10分钟
● P0	SSH 加固 + fail2ban	20分钟
● P0	注册域名 + 提交 ICP 备案	1天提交 + 等待审核

P1	MySQL 自动备份脚本 + 异地 OSS 同步	1小时
P1	接入基础监控 (Netdata + UptimeRobot)	1小时
P1	.NET 加健康检查接口	30分钟
P1	systemd 管理 .NET 服务	30分钟
P1	Nginx 限流 + 安全响应头	30分钟
P1	备案通过后配置 HTTPS	30分钟
P2	Redis 安全加固 (密码 + 禁远程)	15分钟
P2	发布流程标准化 + 软链接回滚	1小时
P2	慢查询优化 + 索引梳理	2-4小时
P2	接入 CDN + WAF	1-2小时
P2	.NET API 限流 + 安全中间件	2-3小时
P3	日志集中收集 + 告警	半天
P3	缓存策略全面梳理	半天
P3	压力测试 + 性能优化	1-2天

最后提醒：运维的核心不是出了事能修好，而是**让事尽量不出**。监控、备份、安全加固，这三件事做扎实了，90% 的故障要么能预防，要么能快速恢复。